

Python cơ bản

Contents

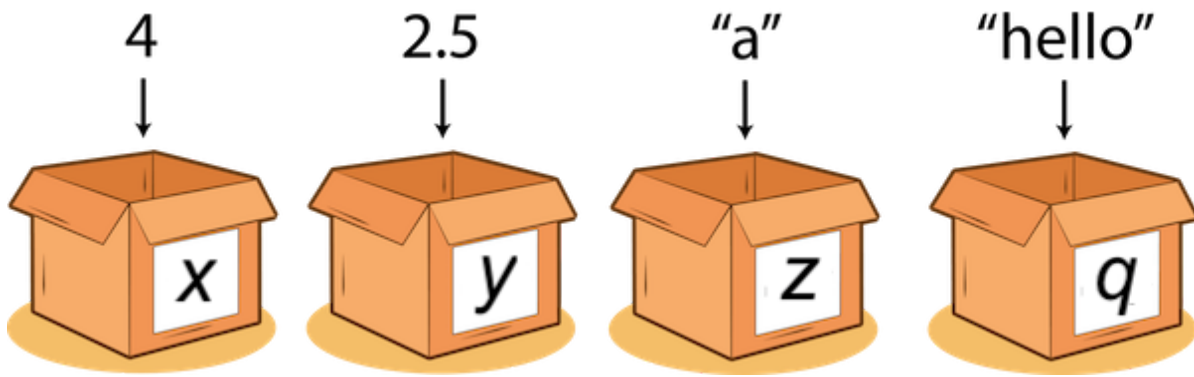
- 4.1. Các kiểu dữ liệu cơ bản trong Python
- 4.2. Cấu trúc dữ liệu tuần tự và tính bất biến
- 4.3. Phương thức chuỗi, từ điển và cấu trúc điều kiện
- 4.4. Câu lệnh điều kiện
- 4.5. Viết vòng lặp
- 4.6. Viết hàm

4.1. Các kiểu dữ liệu cơ bản trong Python

4.1.1. Khái niệm biến

Một **giá trị** là một đơn vị dữ liệu mà chương trình máy tính xử lý, chẳng hạn như một con số hoặc một đoạn văn bản. Có nhiều kiểu giá trị khác nhau: chẳng hạn `4` là một số nguyên và `"Hello!"` là một chuỗi ký tự. Một **biến** là một tên gọi đại diện cho một giá trị. Trong toán học và thống kê, chúng ta thường sử dụng các tên biến như x và y . Trong Python, bạn có thể sử dụng bất kỳ từ nào làm tên biến miễn là tên bắt đầu bằng một chữ cái hoặc dấu gạch dưới. Tuy nhiên, không nên sử dụng từ khóa dành riêng trong Python như `for`, `while`, `class`, `lambda`, v.v., vì đây là những từ mang chức năng đặc biệt trong Python mà chúng ta không nên ghi đè lên.

Bạn có thể hình dung một biến giống như một chiếc hộp chứa thông tin nào đó (một số, một chuỗi, một danh sách, v.v.). Chúng ta sử dụng **toán tử gán** `=` để gán giá trị cho biến.



Hình ảnh được chỉnh sửa từ: medium.com

Các kiểu giá trị của biến, hay gọi tắt là kiểu biến cơ bản có sẵn trong Python bao gồm:

Kiểu biến	Kiểu (python)	Nhóm	Mô tả	Ví dụ
integer	<code>int</code>	Kiểu số	các số nguyên	<code>4</code>
floating point number	<code>float</code>	Kiểu số	các số thực	<code>3.14159</code>
boolean	<code>bool</code>	Kiểu logical	nhận giá trị <code>True</code> hoặc <code>False</code>	<code>True</code>
string	<code>str</code>	Kiểu chuỗi ký tự	Ký tự, chuỗi ký tự, đoạn văn bản	<code>"Khoa học dữ liệu?"</code>
list	<code>list</code>	Kiểu tuần tự (dãy)	một tập hợp các phần tử có thứ tự	<code>['Python', 'R', 'Java']</code>
tuple	<code>tuple</code>	Kiểu tuần tự (dãy)	một tập hợp các phần tử có thứ tự và không thể thay đổi	<code>('Sunday', 6, 3, 2025)</code>
dictionary	<code>dict</code>	Kiểu mapping (ánh xạ)	ánh xạ giữa khóa - giá trị	<code>{'name': 'KHDL', 'code': 123, 'credits': 3}</code>

4.1.2. Dữ liệu kiểu số và toán tử số học

Có biến kiểu số khác nhau trong Python bao gồm số nguyên (integers), số thực (floating point numbers), và số phức (complex numbers). Kiểu số phức là phức tạp nhất và sẽ không được đề cập trong phần này. Để tạo một biến kiểu số, chúng ta chỉ cần viết tên biến và gán bằng giá trị có kiểu thích hợp. Chúng ta có thể xác định kiểu của một đối tượng trong Python bằng cách sử dụng hàm `type()`. Để in giá trị của đối tượng ra màn hình, ta sử dụng hàm `print()`.

```
x = 4
```

```
type(x)
```

```
int
```

```
print(x)
```

```
4
```

Trong Python, dòng lệnh cuối cùng trong một cell sẽ tự động được hiển thị ra màn hình, vì vậy chúng ta không cần phải gọi hàm `print()`.

```
x # Anything after the pound/hash symbol is a comment and will not be run
```

```
4
```

```
pi = 3.14159  
pi
```

```
3.14159
```

```
type(pi)
```

float

Trong Python, các toán tử số học được sử dụng để thực hiện các phép tính giữa các biến kiểu số. Dưới đây là một số toán tử phổ biến:

Toán tử	Mô tả	Ví dụ	Kết quả
<code>+</code>	Cộng	<code>2 + 3</code>	<code>5</code>
<code>-</code>	Trừ	<code>5 - 2</code>	<code>3</code>
<code>*</code>	Nhân	<code>4 * 2</code>	<code>8</code>
<code>/</code>	Chia	<code>6 / 3</code>	<code>2.0</code>
<code>//</code>	Chia lấy phần nguyên	<code>7 // 2</code>	<code>3</code>
<code>%</code>	Chia lấy dư	<code>7 % 2</code>	<code>1</code>
<code>**</code>	Lũy thừa	<code>2 ** 3</code>	<code>8</code>

Bạn có thể sử dụng các toán tử này trực tiếp trong Python để thực hiện phép tính.

```
1 + 2 + 3 + 4 + 5 # cộng
```

15

```
2 * 3.14159 # nhân
```

6.28318

```
2 ** 10 # hàm mũ
```

1024

Phép chia trong Python có thể tạo ra một kiểu dữ liệu khác với kiểu ban đầu mong đợi. Cụ thể phép chia sẽ chuyển giá trị từ kiểu số nguyên (`int`) sang kiểu số thực (`float`).

```
int_2 = 2  
type(int_2)
```

`int`

```
int_2 / int_2 # phép chia
```

`1.0`

```
type(int_2 / int_2)
```

`float`

Toán tử `//` cho phép chúng ta thực hiện phép chia lấy phần nguyên, còn gọi là *floor division*, và giữ nguyên kiểu dữ liệu `int`. Phép toán này luôn làm tròn xuống:

```
101 / 2
```

`50.5`

```
101 // 2 # phép chia làm tròn xuống
```

`50`

Chúng ta gọi đây là phép chia lấy phần nguyên hoặc phép chia làm tròn xuống vì nó giống như việc gọi hàm `int` trên kết quả của một phép chia: giá trị sẽ được làm tròn xuống số nguyên gần nhất:

```
int(101 / 2)
```

50

Toán tử `%`, còn được gọi là **modulo**, trả về phần dư sau khi thực hiện phép chia.

```
100 % 2 # Còn gọi là 100 mod 2
```

0

```
101 % 2 # 101 mod 2
```

1

```
100.5 % 2
```

0.5

4.1.3. Biến kiểu chuỗi ký tự

Ký tự, **chuỗi ký tự**, hay văn bản được lưu trữ dưới dạng một kiểu biến gọi là chuỗi ký tự, hay `string`. Trong nhiều ngôn ngữ lập trình, chuỗi ký tự còn được gọi **dãy các ký tự**.

Tip

Thực ra, để quản lý dữ liệu kiểu chuỗi ký tự trên máy tính điện tử, người ta lưu chúng dưới dạng một chuỗi các mã ký tự Unicode. Bạn đọc có thể tham khảo về [Unicode](#) tại đây. Tương tự như biến kiểu số, để tạo biến dạng chuỗi ký tự, bạn đọc lựa chọn tên biến và gán giá trị kiểu chuỗi ký tự cho biến:

```
my_name = "Trường Công nghệ"
```

```
my_name
```

```
'Trường Công nghệ'
```

```
type(my_name)
```

```
str
```

```
course = 'CT123'
```

```
course
```

```
'CT123'
```

```
type(course)
```

```
str
```

Nếu chuỗi ký tự chứa dấu ngoặc kép `"` hoặc dấu nháy đơn `'`, chúng ta có thể sử dụng kết hợp giữa dấu nháy đơn và dấu ngoặc kép để định nghĩa chuỗi một cách linh hoạt:

```
sentence = "Hôm nay là một ngày nắng nóng đến 40 độ."
```

```
sentence
```

```
'Hôm nay là một ngày nắng nóng đến 40 độ.'
```

```
type(sentence)
```

```
str
```

```
quote = 'Paven: "Cái quý nhất của con người ta là cuộc sống."'
```

```
quote
```

```
'Paven: "Cái quý nhất của con người ta là cuộc sống."'
```

4.1.4. Biến kiểu logical

Biến kiểu logical, hay được gọi trong Python là kiểu `bool` là biến chỉ có hai giá trị là `True` và `False`.

```
the_truth = True
```

```
the_truth
```

```
True
```

```
type(the_truth)
```

```
bool
```

```
lies = False
```

```
lies
```

```
False
```

```
type(lies)
```


bool

Thực tế thì biến kiểu logical được tạo từ kết quả của các toán tử so sánh hơn là được gán trực tiếp từ các giá trị. Chúng ta có thể so sánh các đối tượng trong Python bằng cách sử dụng các **toán tử so sánh**, và kết quả trả về sẽ là một giá trị logical:

Toán tử	Mô tả
<code>x == y</code>	<code>x</code> có bằng <code>y</code> không?
<code>x != y</code>	<code>x</code> có khác <code>y</code> không?
<code>x > y</code>	<code>x</code> có lớn hơn <code>y</code> không?
<code>x >= y</code>	<code>x</code> có lớn hơn hoặc bằng <code>y</code> không?
<code>x < y</code>	<code>x</code> có nhỏ hơn <code>y</code> không?
<code>x <= y</code>	<code>x</code> có nhỏ hơn hoặc bằng <code>y</code> không?
<code>x is y</code>	<code>x</code> có phải là cùng một đối tượng với <code>y</code> ?

```
x = 2 < 3
x
```

True

```
"AI" == "Có thể giải quyết mọi vấn đề"
```

False

```
2 != "2"
```

True

```
2 is 2
```

```
True
```

```
2 == 2.0
```

```
True
```

Chúng ta cũng có các **toán tử logical** dùng để kiểm tra điều kiện, và kết quả trả về cũng là `True` hoặc `False`:

Toán tử	Mô tả
<code>x and y</code>	Cả <code>x</code> và <code>y</code> đều đúng (<code>True</code>) hay không?
<code>x or y</code>	Có ít nhất một trong hai giá trị <code>x</code> hoặc <code>y</code> đúng?
<code>not x</code>	<code>x</code> có sai (<code>False</code>) hay không?

```
True and True
```

```
True
```

```
True and False
```

```
False
```

```
True or False
```

```
True
```

```
False or False
```

```
False
```

```
("Python 2" != "Python 3") and (2 <= 3)
```

```
True
```

```
True
```

```
True
```

```
not True
```

```
False
```

```
not not True
```

```
True
```

Đôi khi chúng ta cần đổi kiểu biến, hay **cast**, một giá trị từ kiểu biến này sang kiểu biến khác một cách tường minh. Điều này có thể thực hiện bằng cách sử dụng các hàm như `str()`, `int()`, và `float()`. Python sẽ cố gắng thực hiện việc chuyển đổi; nếu không thể, kết quả sẽ được trả về là một thông báo lỗi.

```
x = 5.0  
type(x)
```

```
float
```

```
x = int(5.0)
x
```

5

```
type(x)
```

int

```
x = str(5.0)
x
```

'5.0'

```
type(x)
```

str

```
str(5.0) == 5.0
```

False

```
int(5.3)
```

5

```
float("Trường Công nghệ")
```

```
-----  
ValueError                                Traceback (most recent call last)  
Cell In[56], line 1  
----> 1 float("Trường Công nghệ")  
  
ValueError: could not convert string to float: 'Trường Công nghệ'
```

4.2. Cấu trúc dữ liệu tuần tự và tính bất biến

4.2.1. List và tuple

`List` và `tuple` là các đối tượng của Python cho phép chúng ta lưu trữ nhiều giá trị. Các đối tượng này lưu phần tử theo thứ tự, hay **ordered** — chúng ta sẽ tìm hiểu rõ hơn về khái niệm này ở phần sau.

Trước tiên, chúng ta sẽ bắt đầu với `list`. `List` được khởi tạo bằng **dấu ngoặc vuông** `[]`, và bên trong là các phần tử theo thứ tự mà chúng ta muốn khởi tạo:

```
my_list = [1, 2, "ba", 4, 0.5]
```

```
my_list
```

```
[1, 2, 'ba', 4, 0.5]
```

```
type(my_list)
```

```
list
```

`List` là kiểu đối tượng có thể chứa bất kỳ kiểu dữ liệu nào — thậm chí là cả những `list` khác!

```
another_list = [1, "hai", [3, 4, "năm"], True, None, {"key": "value"}]  
another_list
```

```
[1, 'hai', [3, 4, 'năm'], True, None, {'key': 'value'}]
```

Bạn có thể lấy độ dài, hay số phần tử, của một `list` bằng cách sử dụng hàm `len()`:

```
len(my_list)
```

```
5
```

Tương tự như `list`, đối tượng `Tuple` cũng lưu được nhiều phần tử ở nhiều kiểu khác nhau theo thứ tự, nhưng `tuple` có một điểm khác biệt quan trọng: **tuple là đối tượng bất biến**, hay **immutable** — tức là không thể thay đổi sau khi được tạo ra. Chúng ta sẽ tìm hiểu kỹ hơn về vấn đề này ở phần sau. `Tuple` được định nghĩa bằng cách sử dụng **dấu ngoặc tròn** `()` và sau đó liệt kê các đối tượng bên trong:

```
today = (1, 2, "ba", 4, 0.5)
```

```
today
```

```
(1, 2, 'ba', 4, 0.5)
```

```
type(today)
```

```
tuple
```

```
len(today)
```

```
5
```

Chúng ta có thể truy cập các giá trị bên trong một `list`, `tuple` hoặc `string` bằng cú pháp dấu ngoặc vuông `[]`. Python sử dụng **đánh chỉ số bắt đầu từ 0**, nghĩa là phần tử đầu tiên trong dãy nằm ở vị trí 0, không phải vị trí 1.

```
my_list
```

```
[1, 2, 'ba', 4, 0.5]
```

```
my_list[0]
```

```
1
```

```
my_list[2]
```

```
'ba'
```

```
len(my_list)
```

```
5
```

Nếu sử dụng chỉ số vượt quá chiều dài, Python sẽ báo lỗi:

Cell In[70], line 1

Nếu sử dụng chỉ số vượt quá chiều dài, Python sẽ báo lỗi:

^

SyntaxError: invalid syntax

```
my_list[5]
```

```
-----  
IndexError                                Traceback (most recent call last)  
<ipython-input-84-075ca585e721> in <module>  
----> 1 my_list[5]
```

IndexError: list index out of range

Đặc biệt trong Python, chúng ta có thể sử dụng **chỉ số âm** để đếm ngược từ cuối danh sách.

- `-1` là phần tử cuối cùng,
- `-2` là phần tử nằm ngay trước phần tử cuối,

```
my_list
```

```
[1, 2, 'ba', 4, 0.5]
```

```
my_list[-1]
```

```
0.5
```

```
my_list[-2]
```

```
4
```

Chúng ta có thể sử dụng dấu hai chấm `:` để truy cập một phần con của dãy. Cách làm này được gọi là **cắt lát** hay **slicing**.

```
my_list[1:3]
```

```
[2, 'ba']
```

Lưu ý từ ví dụ trên rằng giá trị tương ứng với chỉ số bắt đầu của slice được bao gồm, còn giá trị tương ứng với chỉ số kết thúc thì không. Vì vậy, `my_list[1:3]` sẽ lấy các phần tử ở vị trí 1 và 2, nhưng không lấy phần tử ở vị trí 3.

Chuỗi (`string`) trong Python, có thể được hiểu là một dãy các ký tự cũng hoạt động tương tự như `list` và `tuple`, và hoạt động tương tự khi sử dụng chỉ số và cắt lát.

```
alphabet = "abcdefghijklmnopqrstuvwxyz"
```

```
alphabet[0]
```



```
'a'
```

```
alphabet[-1]
```

```
'z'
```

```
alphabet[-3]
```

```
'x'
```

```
alphabet[:5]
```

```
'abcde'
```

```
alphabet[12:20]
```

```
'mnopqrst'
```

`list` là một đối tượng của Python và nó có các **phương thức** để tương tác với dữ liệu bên trong. Đây là sự khác biệt quan trọng của ngôn ngữ lập trình có hướng đối tượng như Python với các ngôn ngữ không hướng đối tượng.

Có thể hiểu một phương thức giống như một hàm thực hiện một thao tác nào đó trên dữ liệu, nhưng khác với hàm thông thường ở chỗ **phương thức được định nghĩa trên chính đối tượng đó** và được gọi bằng cú pháp dấu chấm `.` ngay sau tên của đối tượng

Ví dụ: `my_list.append(item)` sẽ thêm phần tử `item` vào cuối danh sách có tên `my_list`.

Bạn có thể xem thêm tài liệu hướng dẫn về các [phương thức của list](#).

```
primes = [2, 3, 5, 7, 11]
primes
```

```
[2, 3, 5, 7, 11]
```

```
len(primes)
```

```
5
```

```
primes.append(13)
```

```
primes
```

```
[2, 3, 5, 7, 11, 13]
```

4.2.2. Tập hợp

Một kiểu dữ liệu có sẵn khác trong Python là `set`, dùng để lưu trữ một danh sách các phần tử **không có thứ tự** và các phần tử trong tập hợp phải là **duy nhất**.

Do không có thứ tự, `set` không lưu vị trí phần tử và vì vậy **không hỗ trợ indexing**. Nói cách khác, chúng ta không thể truy cập phần tử bằng chỉ số như `list` hay `tuple`. Tập hợp được khởi tạo bằng cách sử dụng dấu ngoặc nhọn `{}`

```
s = {2, 3, 5, 11}
s
```

```
{2, 3, 5, 11}
```

```
{1, 2, 3} == {3, 2, 1}
```

```
True
```

```
[1, 2, 3] == [3, 2, 1]
```

False

```
s.add(2) # không làm gì cả  
s
```

{2, 3, 5, 11}

```
s[0]
```

```
-----  
TypeError                                Traceback (most recent call last)  
<ipython-input-109-c9c96910e542> in <module>  
----> 1 s[0]  
  
TypeError: 'set' object is not subscriptable
```

Bạn đọc có thể thấy rằng Python sẽ báo lỗi nếu bạn cố gắng truy cập phần tử bằng chỉ số, vì `set` là tập hợp không có thứ tự.

```
### Kiểu dữ liệu có thể thay đổi và không thể thay đổi  
<hr>
```

`String` và `tuple` là các kiểu dữ liệu **không thể thay đổi**, hay *immutable*, trong Python, Nói cách khác, một khi đã được tạo ra, chúng không thể bị sửa đổi.

Ngược lại, `list` là kiểu dữ liệu **có thể thay đổi**, hay *mutable*, vì vậy chúng ta có thể gán lại giá trị mới cho các phần tử trong `list`.

Đây chính là sự khác biệt quan trọng nhất giữa `list` và `tuple`.

```
names_list = ["Nam Định", "Hà Nam", "Ninh Bình"]  
names_list
```

```
['Nam Định', 'Hà Nam', 'Ninh Bình']
```

```
names_list[0] = "Thiên Trường"  
names_list
```

```
['Thiên Trường', 'Hà Nam', 'Ninh Bình']
```

```
names_tuple = ("Nam Định", "Hà Nam", "Ninh Bình")  
names_tuple
```

```
('Nam Định', 'Hà Nam', 'Ninh Bình')
```

```
names_tuple[0] = "Thiên Trường"
```

```
-----  
TypeError                                Traceback (most recent call last)  
<ipython-input-113-bd6a1b77b220> in <module>  
----> 1 names_tuple[0] = "Not cool guy"  
  
TypeError: 'tuple' object does not support item assignment
```

`string` cũng tương tự như `tuple` là một đối tượng bất biến, nghĩa là một khi đã được định nghĩa thì không thể thay đổi các ký tự bên trong. Điều này có nghĩa là chúng ta không thể gán lại giá trị cho một vị trí cụ thể trong chuỗi sau khi nó đã được tạo ra.

```
my_name = "Trường Công nghệ"
```

```
my_name[1] = "t"
```

```
-----  
TypeError                                Traceback (most recent call last)  
Cell In[28], line 1  
----> 1 my_name[1] = "t"  
  
TypeError: 'str' object does not support item assignment
```

```
x = ([1, 2, 3], 5)
```

```
x[1] = 7
```

```
-----  
TypeError                                Traceback (most recent call last)  
<ipython-input-117-415ce6bd0126> in <module>  
----> 1 x[1] = 7  
  
TypeError: 'tuple' object does not support item assignment
```

```
x
```

```
([1, 2, 3], 5)
```

```
x[0][1] = 4
```

```
x
```

```
([1, 4, 3], 5)
```

4.3. Phương thức chuỗi, từ điển và cấu trúc điều kiện

4.3.1. Các phương thức của chuỗi

Trong Python, chuỗi ký tự là một kiểu dữ liệu đối tượng và có nhiều **phương thức tích hợp sẵn** để thao tác với dữ liệu văn bản. Các phương thức này được gọi bằng cú pháp dấu chấm, ví dụ:

```
my_string.method_name().
```

Dưới đây là một số phương thức thường dùng trên chuỗi ký tự:

Phương thức	Mô tả	Ví dụ
<code>lower()</code>	Chuyển tất cả ký tự thành chữ thường	<code>"Hello".lower()</code> → <code>'hello'</code>
<code>upper()</code>	Chuyển tất cả ký tự thành chữ hoa	<code>"abc".upper()</code> → <code>'ABC'</code>
<code>strip()</code>	Loại bỏ khoảng trắng đầu và cuối chuỗi	<code>" hello ".strip()</code> → <code>'hello'</code>
<code>replace(a, b)</code>	Thay tất cả <code>a</code> bằng <code>b</code>	<code>"cat".replace("c", "b")</code> → <code>'bat'</code>
<code>split()</code>	Tách chuỗi thành danh sách dựa trên khoảng trắng (hoặc ký tự cho trước)	<code>"a b c".split()</code> → <code>['a', 'b', 'c']</code>
<code>join(list)</code>	Ghép danh sách chuỗi thành một chuỗi, dùng ký tự nối	<code>" ".join(['a', 'b'])</code> → <code>'a b'</code>
<code>find(sub)</code>	Trả về vị trí đầu tiên của chuỗi con <code>sub</code> (nếu có)	<code>"hello".find("e")</code> → <code>1</code>

■ Xem thêm tài liệu về các phương thức chuỗi tại: [Python String Methods](#)

Dưới đây là một số ví dụ sử dụng phương thức chuỗi:

```
all_caps = "XIN CHÀO NEUTECH?"
all_caps
```

```
'XIN CHÀO NEUTECH?'
```

```
new_str = all_caps.lower()
new_str
```

```
'xin chào neutech?'
```

Lưu ý rằng phương thức `lower()` không thay đổi chuỗi ban đầu, mà thay vào đó **trả về một chuỗi mới** với các ký tự đã được chuyển thành chữ thường. Nhắc lại rằng chuỗi trong Python là

kiểu **bất biến**, vì vậy mọi thao tác biến đổi đều tạo ra một đối tượng chuỗi mới.

```
all_caps
```

```
'XIN CHÀO NEUTECH?'
```

```
all_caps.split()
```

```
['XIN', 'CHÀO', 'NEUTECH?']
```

```
all_caps.count("0")
```

```
1
```

Chúng ta cũng có thể **kết hợp nhiều phương thức với nhau**, còn gọi là *method chaining*. Điều này cho phép thực hiện nhiều thao tác liên tiếp trên một đối tượng. Chúng ta sẽ còn gặp lại khái niệm này nhiều hơn khi học về thư viện NumPy và Pandas ở các chương sau.

Python cung cấp nhiều cách để tạo chuỗi bằng cách **điền vào chỗ trống** và định dạng chúng một cách gọn gàng. Điều này rất hữu ích khi bạn muốn tạo các chuỗi có chứa biến hoặc kết quả tính toán từ các bước trước đó.

Có một số cách thực hiện việc này, nhưng phương pháp được khuyến nghị sử dụng là **f-strings**. Với f-strings, bạn đọc chỉ cần thêm chữ cái **f** ở đầu chuỗi, và chèn biến trực tiếp vào trong **ngoặc nhọn** **{ }**. Hãy quan sát ví dụ sau:

```
name = "NeuTech"
age = 16/12
day = 16
month = 12
year = 2023
template_new = f"Xin chào, tôi là {name}. Tôi {age:.2f} tuổi. Tôi sinh ngày {day}/{mon
template_new
```

'Xin chào, tôi là NeuTech. Tôi 1.33 tuổi. Tôi sinh ngày 16/12/2023.'

Note

Trong đoạn code trên, ký hiệu sau dấu hai chấm bên trong dấu ngoặc nhọn `{}` là dùng để định dạng. Ví dụ, `:.2f` nghĩa là: in biến này với **2 chữ số thập phân**.

Bạn có thể xem thêm các tùy chọn định dạng chuỗi tại [đây](#)

4.3.2. Từ điển

Một từ điển, hay **dictionary**, là một cấu trúc ánh xạ giữa các **cặp khóa–giá trị** và được định nghĩa bằng dấu ngoặc nhọn `{}`, và được khai báo như sau:

```
house = {
    "Phòng ngủ": 5,
    "Phòng vệ sinh": 5,
    "Thành phố": "Hà Nội",
    "Giá": "15 tỷ",
    "Ngày bán": (1, 3, 2025),
}

condo = {
    "Phòng ngủ": 3,
    "Phòng vệ sinh": 2,
    "Thành phố": "Hồ Chí Minh",
    "Giá": "5 tỷ",
    "Ngày bán": (1, 5, 2025),
}
```

Chúng ta có thể truy cập một trường cụ thể trong **dictionary** bằng cách sử dụng **dấu ngoặc vuông** `[]`, với **tên khóa** bên trong.

```
house["Giá"]
```

```
'15 tỷ'
```

```
condo["Thành phố"]
```



```
'Hồ Chí Minh'
```

Chúng ta cũng có thể chỉnh sửa từ điển vì đây là kiểu dữ liệu **có thể thay đổi**.

```
condo["Giá"] = "6 tỷ" # price already in the dict  
condo
```

```
{'Phòng ngủ': 3,  
 'Phòng vệ sinh': 2,  
 'Thành phố': 'Hồ Chí Minh',  
 'Giá': '6 tỷ',  
 'Ngày bán': (1, 5, 2025)}
```

```
condo["Sàn nhà"] = "Lát gỗ"  
house["Sàn nhà"] = "Gạch"
```

```
condo
```

```
{'Phòng ngủ': 3,  
 'Phòng vệ sinh': 2,  
 'Thành phố': 'Hồ Chí Minh',  
 'Giá': '6 tỷ',  
 'Ngày bán': (1, 5, 2025),  
 'Sàn nhà': 'Lát gỗ'}
```

Hoặc chúng ta cũng có thể sử dụng chỉ số để thêm thông tin vào trong từ điển

```
condo[5] = 123
```

```
condo
```

```
{'bedrooms': 2,  
 'bathrooms': 1,  
 'price': 5,  
 'date_sold': (27, 8, 2011),  
 'flooring': 'wood',  
 5: 443345}
```

Khóa (**keys**) trong từ điển có thể là bất kỳ kiểu dữ liệu bất biến nào, thậm chí là một **tuple**!

```
house[("Tầng 1", "Tầng 2", "Tầng 3")] = (100, 80, 80)
house
```

```
{'Phòng ngủ': 5,
 'Phòng vệ sinh': 5,
 'Thành phố': 'Hà Nội',
 'Giá': '15 tỷ',
 'Ngày bán': (1, 3, 2025),
 'Sàn nhà': 'Gạch',
 ('Tầng 1', 'Tầng 2', 'Tầng 3'): (100, 80, 80)}
```

Python sẽ báo lỗi nếu chúng ta cố gắng truy cập vào một key không tồn tại

```
house["Tầng 1"]
```

```
-----
KeyError                                Traceback (most recent call last)
Cell In[44], line 1
----> 1 house["Tầng 1"]

KeyError: 'Tầng 1'
```

4.3.3. Đối tượng rỗng

Đôi khi chúng ta có thể cần tạo ra các đối tượng rỗng để sau đó điền giá trị vào. Cách tạo ra đối tượng rỗng của một số kiểu dữ liệu như sau

```
lst = list() # một list trống
lst
```

```
[]
```

```
lst = [] # cho kết quả tương tự
lst
```

```
[]
```

Không có sự khác biệt thực sự giữa hai cách khởi tạo danh sách ở trên. Tuy nhiên, cú pháp `[]` được cho là **nhANH hơn một chút** so với `list()` trong một số trường hợp nhất định. Để tạo một `tuple`, hay từ điển rỗng, chúng ta khai báo tương tự

```
tup = tuple() # empty tuple  
tup
```

```
()
```

```
tup = () # empty tuple  
tup
```

```
()
```

```
dic = dict() # empty dict  
dic
```

```
{}
```

```
dic = {} # empty dict  
dic
```

```
{}
```

```
st = set() # empty set  
st
```

```
set()
```

4.4. Câu lệnh điều kiện

Câu lệnh điều kiện cho phép chúng ta viết các chương trình mà trong đó chỉ những khối câu lệnh nhất định được thực thi, tùy thuộc vào trạng thái hiện tại của chương trình.

Hãy cùng xem một số ví dụ và chú ý đến các **từ khóa**, **cú pháp**, và **cách thụt đầu dòng (indentation)** – một phần rất quan trọng trong Python để xác định phạm vi của các khối câu lệnh.

```
name = "NeuTech"

if name.lower() == "neutech":
    print("That's my name too!")
elif name.lower() == "santa":
    print("That's a funny name.")
else:
    print(f"Hello {name}! That's a cool name!")
print("Nice to meet you!")
```

```
That's my name too!
Nice to meet you!
```

Những điểm cần lưu ý:

- Sử dụng các từ khóa `if`, `elif` và `else` để viết các câu lệnh điều kiện.
- Dấu hai chấm `:` kết thúc mỗi biểu thức điều kiện.
- Thụt đầu dòng: mặc định 4 dấu cách được sử dụng để xác định khối lệnh thuộc về điều kiện.
- Trong một câu lệnh `if`, khối câu lệnh đầu tiên có điều kiện trả về `True` sẽ được thực thi, và chương trình sẽ thoát khỏi toàn bộ cấu trúc `if`.
- Câu lệnh `if` không bắt buộc phải có `elif` hay `else`.
- `elif` cho phép kiểm tra thêm nhiều điều kiện khác nhau.
- `else` cho phép thực hiện một khối mặc định nếu tất cả các điều kiện khác đều sai.
- Phần kết thúc của toàn bộ cấu trúc `if` ** là nơi thụt đầu dòng cùng mức với từ khóa `if` ban đầu.

Trong các đoạn lệnh phức tạp, câu lệnh điều kiện có thể nằm trong một câu lệnh điều kiện khác:

```

name = "Super NeuTech"

if name.lower() == "neutech":
    print("That's my name too!")
elif name.lower() == "santa":
    print("That's a funny name.")
else:
    print(f"Hello {name}! That's a cool name.")
    if name.lower().startswith("super"):
        print("Do you really have superpowers?")

print("Nice to meet you!")

```

```

Hello Super NeuTech! That's a cool name.
Do you really have superpowers?
Nice to meet you!

```

Chúng ta có thể viết các câu lệnh `if` đơn giản trên một dòng để tăng tính ngắn gọn và đơn giản của code.

```

words = ["the", "list", "of", "words"]

x = "long list" if len(words) > 10 else "short list"
x

```

```
'short list'
```

```

if len(words) > 10:
    x = "long list"
else:
    x = "short list"

```

```
x
```

```
'short list'
```

Trong Python, **mọi đối tượng** đều có thể được kiểm tra tính “đúng” hay “sai” để sử dụng trong các câu lệnh điều kiện như `if`.

- **Giá trị `True`**: Mặc định, tất cả các đối tượng đều được coi là `True`, **trừ khi** chúng là đối tượng `bool` với giá trị `False`, hoặc có độ dài `len()` bằng 0.
- **Giá trị `False`**: bao gồm các giá trị đặc biệt sau:
 - `None`
 - `False`
 - Số 0
 - Các chuỗi và cấu trúc dữ liệu rỗng: `''`, `()`, `[]`, `{}`, `set()`

Tip

Xem chi tiết thêm tại [tài liệu chính thức của Python](#).

```
x = 'a'

if x:
    print("Tôi là True!")
else:
    print("Tôi là False!")
```

Tôi là True!

```
x = 0.0

if x:
    print("Tôi là True!")
else:
    print("Tôi là False!")
```

Tôi là False!

```
x = []

if x:
    print("Tôi là True!")
else:
    print("Tôi là False!")
```

4.5. Viết vòng lặp

4.5.1. Vòng lặp `for`

Vòng lặp `for` trong Python được sử dụng để thực thi một đoạn code lặp lại nhiều lần, với số lần lặp được xác định rõ ràng, thường dựa trên một dãy số, một chuỗi, một danh sách hoặc một đối tượng có thể lặp. Đây là một cấu trúc điều khiển quan trọng giúp tự động hóa các thao tác lặp lại thay vì viết thủ công nhiều lần.

```
for n in [1, 3, -1, 5]:  
    print(f"Số được chọn là {n} và bình phương của nó là {n**2}")  
print("Kết thúc vòng lặp")
```

```
Số được chọn là 1 và bình phương của nó là 1  
Số được chọn là 3 và bình phương của nó là 9  
Số được chọn là -1 và bình phương của nó là 1  
Số được chọn là 5 và bình phương của nó là 25  
Kết thúc vòng lặp
```

Khi viết vòng lặp `for` cần lưu ý:

- Từ khóa `for` bắt đầu vòng lặp, và dấu hai chấm `:` kết thúc dòng khai báo đầu tiên.
- Khối lệnh thụt đầu dòng sẽ được thực thi cho mỗi giá trị trong đối tượng lặp (*iterable*).
- Vòng lặp sẽ kết thúc sau khi biến lặp (ví dụ: `n`) lần lượt nhận tất cả các giá trị trong dãy.
- Có thể lặp qua bất kỳ đối tượng nào là **iterable**, bao gồm: `list`, `tuple`, `range`, `set`, `string`.
- Một **iterable** là bất kỳ đối tượng nào chứa một tập hợp giá trị có thể lặp tuần tự.

```
word = "Trường Công nghệ"  
for letter in word:  
    print("Ký tự hiện tại là: " + letter)  
  
print(f"Tên đầy đủ là {word}!")
```

```
Chữ cái hiện tại là: T
Chữ cái hiện tại là: r
Chữ cái hiện tại là: ư
Chữ cái hiện tại là: ờ
Chữ cái hiện tại là: n
Chữ cái hiện tại là: g
Chữ cái hiện tại là:
Chữ cái hiện tại là: C
Chữ cái hiện tại là: ô
Chữ cái hiện tại là: n
Chữ cái hiện tại là: g
Chữ cái hiện tại là:
Chữ cái hiện tại là: n
Chữ cái hiện tại là: g
Chữ cái hiện tại là: h
Chữ cái hiện tại là: ệ
Tên đầy đủ là Trường Công nghệ!
```

Một đối tượng lặp rất phổ biến trong Python là sử dụng vòng lặp `for` kết hợp với hàm `range()`. Hàm `range()` tạo ra một dãy số nguyên tuần tự, bắt đầu từ một giá trị và kết thúc **không bao gồm** giá trị cuối cùng được chỉ định. Cách sử dụng này đặc biệt hữu ích khi bạn cần lặp lại một hành động một số lần cụ thể, chẳng hạn như:

```
for i in range(10):
    print(i)
```

Hàm `range()` còn có thể tạo ra một dãy các giá trị của một cấp số cộng hoặc trừ, bắt đầu từ một số, kết thúc ở 1 số khác.

```
for i in range(1, 101, 10): # Bắt đầu từ 1, không vượt quá 100, tăng đều 10
    print(i)
```

Chúng ta có thể viết một vòng lặp bên trong một vòng lặp khác để lặp qua nhiều dãy giá trị khác nhau:

```
for x in [1, 2, 3]:
    for y in ["a", "b", "c"]:
        print((x, y))
```



```
(1, 'a')
(1, 'b')
(1, 'c')
(2, 'a')
(2, 'b')
(2, 'c')
(3, 'a')
(3, 'b')
(3, 'c')
```

```
list_1 = [0, 1, 2]
list_2 = ["a", "b", "c"]
for i in range(3):
    print(list_1[i], list_2[i])
```

```
0 a
1 b
2 c
```

Khi lặp qua các đối tượng, người lập trình thường có xu hướng sử dụng `zip()` và `enumerate()`. `zip()` trả về một đối tượng `zip`, là một đối tượng lặp được của các `tuple`. `enumerate()` thêm một bộ đếm vào một đối tượng lặp được mà chúng ta có thể sử dụng trong vòng lặp.

```
for i in zip(list_1, list_2):
    print(i)
```

```
(0, 'a')
(1, 'b')
(2, 'c')
```

Chúng ta thậm chí có thể giải nén các `tuple` này trực tiếp trong vòng lặp `for`:

```
for i, j in zip(list_1, list_2):
    print(i, j)
```

```
0 a
1 b
2 c
```

```
for i in enumerate(list_2):  
    print(i)
```

```
(0, 'a')  
(1, 'b')  
(2, 'c')
```

```
for n, i in enumerate(list_2):  
    print(f"index {n}, value {i}")
```

```
index 0, value a  
index 1, value b  
index 2, value c
```

Chúng ta có thể lặp qua các cặp khóa-giá trị của một từ điển bằng cách sử dụng `.items()`. Cú pháp chung là `for key, value in dictionary.items()`.

```
courses = {123 : "thú vị",  
            231 : "bình thường",  
            312 : "buồn ngủ!"}  
  
for course_num, description in courses.items():  
    print(f"NCT {course_num} thì {description}")
```

```
NCT 123 thì thú vị  
NCT 231 thì bình thường  
NCT 312 thì buồn ngủ!
```

Chúng ta thậm chí có thể sử dụng `enumerate()` để thực hiện việc giải nén phức tạp hơn:

```
for n, (course_num, description) in enumerate(courses.items()):  
    print(f"Môn học {n+1}: NCT {course_num} thì {description}")
```

```
Môn học 1: NCT 123 thì thú vị  
Môn học 2: NCT 231 thì bình thường  
Môn học 3: NCT 312 thì buồn ngủ!
```

4.5.2. Vòng lặp `while`

Chúng ta cũng có thể sử dụng một [vòng lặp `while`](#) để thực thi một khối câu lệnh được nhiều lần. Cách viết vòng lặp `while` là `while` + “biểu thức điều kiện” + “khối lệnh được lặp”. Khi biểu thức điều kiện cho giá trị là `True` thì các khối lệnh sẽ được thực thi. Cần lưu ý là khi biểu thức điều kiện luôn là `True`, thì bạn sẽ có một vòng lặp vô hạn và đoạn lệnh sẽ không bao giờ dừng lại.

```
n = 5
while n > 0:
    print(n)
    n -= 1
print("Done!")
```

```
10
9
8
7
6
5
4
3
2
1
Blast off!
```

Bạn đọc có thể hình dung câu lệnh `while` giống như đọc ngữ pháp thông thường: “Khi `n` lớn hơn 0, hiển thị giá trị của `n` và sau đó giảm `n` đi 1. Khi `n` bằng 0, biểu thức điều kiện sai và kết thúc vòng lặp, sau đó hiển thị từ Done!”

Trong nhiều trường hợp, bạn đọc có thể muốn buộc vòng lặp `while` dừng lại dựa trên một số tiêu chí bằng cách sử dụng từ khóa `break`.

```

n = 123
i = 0
while n != 1:
    print(int(n))
    if n % 2 == 0: # n is even
        n = n / 2
    else: # n is odd
        n = n * 3 + 1
    i += 1
    if i == 10:
        print(f"Đã lặp quá nhiều!")
        break

```

```

123
370
185
556
278
139
418
209
628
314
Đã lặp quá nhiều!

```

4.5.3. Viết vòng lặp sử dụng tạo danh sách rút gọn

Tạo danh sách rút gọn cho phép chúng ta khởi tạo `list`, `set`, `dictionary` trong một dòng lệnh một cách tiện lợi, nhỏ gọn. Dưới đây là một vòng lặp `for` tiêu chuẩn mà bạn có thể sử dụng để lặp qua một đối tượng lặp được và tạo một danh sách. Hãy quan sát ví dụ dưới đây, khi chúng ta muốn lấy ra chữ cái đầu tiên của các từ trong 1 list, cách viết vòng lặp chuẩn như sau:

```

mylist = ['Trường', 'Công', 'nghệ', 'là', 'đơn', 'vị', 'đào', 'tạo', 'ứng', 'dụng', 'c']
first_letters = []
for word in mylist:
    first_letters.append(word[0])
print(first_letters)

```

```
['T', 'C', 'n', 'l', 'đ', 'v', 'đ', 't', 'ứ', 'd', 'c', 'n', 't', 'k', 't']
```

Tạo danh sách rút gọn cho phép chúng ta thực hiện điều này chỉ trong một dòng lệnh nhỏ gọn hơn nhiều:

```
[word[0] for word in mylist]
```

```
['T', 'C', 'n', 'l', 'đ', 'v', 'đ', 't', 'ứ', 'd', 'c', 'n', 't', 'k', 't']
```

Chúng ta thậm chí có thể thực hiện các vòng lặp lồng nhau khi tạo danh sách rút gọn, hoặc sử dụng kèm với biểu thức điều kiện:

```
[(i, j) for i in range(3) for j in range(4)]
```

```
[(0, 0),  
(0, 1),  
(0, 2),  
(0, 3),  
(1, 0),  
(1, 1),  
(1, 2),  
(1, 3),  
(2, 0),  
(2, 1),  
(2, 2),  
(2, 3)]
```

```
[i for i in range(11) if i % 2 == 0] # condition the iterator, select only even number
```

```
[0, 2, 4, 6, 8, 10]
```

```
[-i if i % 2 else i for i in range(11)] # condition the value, -ve odd and +ve even number
```

```
[0, -1, 2, -3, 4, -5, 6, -7, 8, -9, 10]
```

Bạn đọc cũng có thể sử dụng cách viết này với các tập hợp. Lưu ý rằng tập hợp không có thứ tự phần tử và sẽ loại đi các phần tử bị lặp:

```
{word[0] for word in mylist}
```

```
{'C', 'T', 'c', 'd', 'k', 'l', 'n', 't', 'v', 'đ', 'ứ'}
```

Danh sách rút gọn cũng có thể áp dụng cho từ điển:

```
word_lengths = {word:len(word) for word in mylist} # dictionary comprehension  
word_lengths
```

```
{'Trường': 6,  
 'Công': 4,  
 'nghệ': 4,  
 'là': 2,  
 'đơn': 3,  
 'vị': 2,  
 'đào': 3,  
 'tạo': 3,  
 'ứng': 3,  
 'dụng': 4,  
 'công': 4,  
 'trong': 5,  
 'kinh': 4,  
 'tế': 2}
```

4.6. Viết hàm

Một [hàm](#) là một đoạn câu lệnh được viết để tái sử dụng, có thể chấp nhận các tham số đầu vào, còn được gọi là đối số hoặc tham số (arguments). Ví dụ, để định nghĩa một hàm có tên `square` nhận một tham số đầu vào `n` và trả về bình phương `n**2`, chúng ta khai báo như sau:

```
def square(n):  
    n_squared = n**2  
    return n_squared
```

```
square(2)
```

```
square(100)
```

```
10000
```

```
square(12345)
```

```
152399025
```

Hàm bắt đầu bằng từ khóa `def`, sau đó là tên hàm, các tham số trong dấu ngoặc đơn, và sau đó là dấu hai chấm (`:`). Các câu lệnh được thực thi bởi hàm được xác định bằng cách viết thụt lề. Giá trị đầu ra hoặc giá trị “trả về” của hàm được chỉ định bằng từ khóa `return`.

4.6.1. Tác dụng phụ & biến cục bộ

Khi bạn tạo một biến bên trong một hàm, biến đó là **biến cục bộ**, có nghĩa là nó chỉ tồn tại bên trong hàm đó. Ví dụ:

```
def cat_string(str1, str2):  
    string = str1 + str2  
    return string
```

```
cat_string('Tôi là ', 'NeuTech')
```

```
'Tôi là NeuTech'
```

```
string
```

```
-----  
NameError                                Traceback (most recent call last)  
Cell In[11], line 1  
----> 1 string  
  
NameError: name 'string' is not defined
```

Bạn đọc có thể thấy rằng biến `string` có được truyền giá trị bên trong hàm `cat_string` nhưng khi gọi ở môi trường bên ngoài thì lại báo lỗi. Điều đó thể hiện là biến `string` chỉ là một biến cục bộ.

Nếu một hàm thay đổi các biến được truyền vào nó, thì nó được cho là có **tác dụng phụ**, hay side effects. Ví dụ:

```
def silly_sum(my_list):  
    my_list.append(0)  
    return sum(my_list)
```

```
l = [1, 2, 3, 4]  
out = silly_sum(l)  
out
```

10

Trong ví dụ trên, ngoài kết quả cho tổng bằng 10, hàm còn thay đổi đối tượng `l`:

```
l
```

[1, 2, 3, 4, 0]

Nếu hàm có các tác dụng phụ như thế này, người viết cần phải đề cập đến yếu tố này trong tài liệu cho người dùng.

Nếu bạn không chỉ định giá trị trả về, hàm sẽ trả về `None` khi nó kết thúc:

```
def f(x):  
    x + 1 # no return!  
    if x == 999:  
        return  
print(f(0))
```

None

4.6.2. Tham số tùy chọn và bắt buộc

Khi viết hàm số có tham số, chúng ta thường gán *giá trị mặc định* cho một số tham số trong một hàm, các tham số này còn được gọi là “tham số tùy chọn”, nghĩa là không nhất thiết phải gọi các tham số này mỗi khi gọi hàm. Khi tham số không được gọi, giá trị mặc định sẽ được sử dụng. Chỉ khi muốn truyền giá trị khác cho tham số thì người sử dụng hàm mới cần gọi tham số:

```
def repeat_string(s, n=2):  
    return s*n
```

```
mystr = "Don't quit; "  
repeat_string(mystr) # Không cần khai báo n nếu không muốn thay đổi
```

```
"Don't quit; Don't quit; "
```

```
repeat_string(mystr, 3) # Truyền giá trị khác nếu muốn thay đổi
```

```
'mdsmdsmdsmdsm'
```

Khi khai báo hàm số, giá trị mặc định cho các tham số tùy chọn nên được chọn cẩn thận. Khi viết hàm số bao gồm cả các tham số tùy chọn và tham số bắt buộc, cần lưu ý là:

- Tất cả các tham số tùy chọn phải đứng sau các tham số bắt buộc.
- Các tham số bắt buộc được ánh xạ theo thứ tự chúng xuất hiện.
- Các tham số tùy chọn có thể được chỉ định không theo thứ tự khi sử dụng hàm.

```
def example(a = 1, b, c = -1, d = -2): # Báo lỗi nếu khai báo tham số tùy chọn trước  
    print(a, b, c, d)
```

```
example(1, 2, 3, 4) # Truyền giá trị cho cả 4 tham số
```

```
Cell In[27], line 1
    def example(a = 1, b, c = -1, d = -2): # Báo lỗi nếu khai báo tham số tùy chọn trước
        ^
SyntaxError: parameter without a default follows parameter with a default
```

```
def example(a, b, c = -1, d = -2): # Tham số tùy chọn được khai báo sau
    print(a, b, c, d)

example(1, 2, 3, 4) # Truyền giá trị cho cả 4 tham số
```

```
1 2 3 4
```

Sử dụng các giá trị mặc định cho `c` và `d`:

```
example(1, 2)
```

```
1 2 -1 -2
```

Có thể chỉ chỉ định chỉ một trong các tham số tùy chọn:

```
example(1, 2, c=3)
```

```
1 2 3 -2
```

Chỉ định tất cả các đối số làm tham số từ khóa, mặc dù chỉ có `c` và `d` là tùy chọn:

```
example(a=1, b=2, c=3, d=4)
```

```
1 2 3 4
```

Chỉ định các tham số tùy chọn theo từ khóa, nhưng sai thứ tự, mặc dù cũng có thể gây nhầm lẫn, nhưng vẫn cho kết quả đúng:

```
example(1, 2, d=4, c=3)
```

```
1 2 3 4
```

4.6.3. Hàm số trả về nhiều giá trị

Trong đa số ngôn ngữ lập trình, hàm chỉ có thể trả về một đối tượng. Về mặt kỹ thuật hàm trong Python cũng chỉ trả về một đối tượng. Để giải quyết vấn đề này chúng ta sẽ thiết kế hàm trả về một `tuple`, một đối tượng có thể chứa được nhiều giá trị:

```
def sum_and_product(x, y):  
    return (x + y, x * y)
```

```
sum_and_product(5, 6)
```

```
(11, 30)
```

Dấu ngoặc đơn có thể được bỏ qua và một `tuple` được trả về một cách ngầm định được xác định bằng việc sử dụng dấu phẩy:

```
def sum_and_product(x, y):  
    return x + y, x * y
```

```
sum_and_product(5, 6)
```

```
(11, 30)
```

Thông thường, chúng ta sẽ giải nén một `tuple` trả về thành các biến riêng biệt, điều đó tạo cảm giác như hàm đang trả về nhiều giá trị:

```
s, p = sum_and_product(5, 6)
```

s

11

p

30

Ngoài ra, theo quy ước trong Python, chúng ta sử dụng biến có tên `_` cho các giá trị mà bạn không muốn lưu:

```
s, _ = sum_and_product(5, 6)
```

s

11

_

30

Bạn cũng có thể định nghĩa các hàm chấp nhận một số lượng tùy ý các tham số vị trí hoặc từ khóa bằng cách sử dụng `*args` và `**kwargs`.

```
def add(*args):  
    print(args)  
    return sum(args)
```

```
add(1, 2, 3, 4, 5, 6)
```

(1, 2, 3, 4, 5, 6)

21

```
def add(**kwargs):  
    print(kwargs)  
    return sum(kwargs.values())
```

```
add(a=3, b=4, c=5)
```

```
{'a': 3, 'b': 4, 'c': 5}
```

12

4.6.4. Hàm ẩn

Có hai cách để định nghĩa hàm trong Python. Cách mà chúng ta đã sử dụng là sử dụng từ khóa

`def`

```
def add_one(x):  
    return x+1
```

```
add_one(9.75)
```

8.2

Có một cách khác để khai báo hàm đó là sử dụng từ khóa `lambda`:

```
add_one = lambda x: x+1
```

```
type(add_one)
```

```
function
```

```
add_one(9.75)
```

```
10.75
```

Hai cách tiếp cận trên là cho kết quả giống nhau. Cách sử dụng từ khóa `lambda` được gọi là **hàm ẩn**, hay anonymous function). Hàm ẩn chỉ có thể viết trên một dòng lệnh, vì vậy chúng không phù hợp trong các trường hợp yêu cầu định nghĩa hàm phức tạp, nhưng có thể hữu ích trong một số trường hợp khác, đặc biệt là khi muốn viết câu lệnh ngắn gọn, hoặc muốn viết một hàm nằm trong một hàm khác:

```
def power_function(n):  
    # Trả về một hàm lambda, hàm này nhận x và trả về x^n  
    return lambda x: x ** n  
  
# Tạo một hàm lập phương  
cube = power_function(3)  
cube(2)
```

```
8
```

Đoạn lệnh ở trên có nghĩa là

- Trước tiên, `lambda x: x**n` đánh giá thành một giá trị thuộc kiểu `function`, là giá trị trả lại trong hàm `power_function`.
- Chúng ta muốn định nghĩa một hàm bậc 3 (lập phương) thì chỉ cần gọi tham số 3 trong hàm `power_function`
- Hàm `cube` được hiểu là lập phương của một giá trị đầu vào, và cho kết quả tại 2 là 8

Như vậy, trong chương này, chúng ta đã được giới thiệu tương đối đầy đủ về kiến thức nền tảng của ngôn ngữ Python. Các chương tiếp theo sẽ thảo luận về các thư viện quan trọng của Python trong xử lý và phân tích dữ liệu, bao gồm `numpy`, `pandas`, và `matplotlib`.